

Coin Dash Pt. 1

Prepared by: Rene Andre Bedonia Jocsing

Published: January 30, 2026

Session Objectives

- Understand Area2D-based collision detection
- Implement input-driven movement with proper delta timing
- Build reusable, testable game objects through scene composition
- Use signals for decoupled communication

Project Setup

1. Create New Project

1. Open Godot, click New Project
2. Name: CoinDash
3. Create project folder
4. Click Create & Edit

2. Configure Window Settings

Project, Project Settings, Display, Window:

```
1 Viewport Width: 480
2 Viewport Height: 720
3 Stretch/Mode: canvas_items
4 Stretch/Aspect: keep
5 Size/Resizable: OFF
```

3. Import Assets

- Download coin_dash_assets.zip
- Unzip into project folder
- Verify res://assets/ contains player, coin, audio folders

Common Pitfalls

Window too small? Zoom viewport using View, Zoom In (Ctrl+Plus)

Assets not showing? Check FileSystem panel, right-click, Reimport

Part 1: Player Scene Architecture

Design Decision: Why Area2D?

We use Area2D as the root because the player needs:

- Collision detection (not physics response)
- Signal-based communication when overlapping coins/obstacles
- Manual movement control (not rigid body simulation)

Scene Tree Structure

```
1 Player (Area2D)
2 |-- AnimatedSprite2D
3 |-- CollisionShape2D
```

Step-by-Step Build

1. Create Root Node

- Click Add/Create New Node (Ctrl+A)
- Search: Area2D
- Rename to Player
- Save scene (Ctrl+S): `player.tscn`

2. Lock Children (CRITICAL)

- Select Player node
- Click icon next to lock: Group Selected Node(s)
- Tooltip: Make selected node's children not selectable
- Icon appears next to Player name

Common Pitfalls

Forgetting to group children causes:

- Accidentally moving/scaling child nodes
- Misaligned collision shapes
- Hard-to-debug position offsets

Always group immediately after creating root!

3. Add AnimatedSprite2D

- Select Player, Add Child Node, AnimatedSprite2D
- Inspector, Frames: empty, New SpriteFrames
- Click the SpriteFrames text to open panel at bottom

4. Configure Animations

In SpriteFrames panel (bottom):

1. Rename default animation to run
2. Click Add Animation, create idle
3. Click Add Animation, create hurt
4. For each animation:
 - Navigate to `res://assets/player/` in FileSystem
 - Drag corresponding images into animation frames
 - Set Speed: 8 FPS
5. Select idle, click Autoplay on Load (play icon)

5. Scale Sprite

- Select `AnimatedSprite2D`
- Inspector, Transform, Scale: (2, 2)

6. Add Collision Shape

- Select Player, Add Child, `CollisionShape2D`
- Inspector, Shape: empty, New `RectangleShape2D`
- Drag orange handles to cover sprite
- Pro tip: Hold Alt while dragging for symmetric sizing

7. Adjust Sprite Offset

- Select `AnimatedSprite2D`
- Inspector, Offset: (0, -5)
- Reason: Centers collision shape on sprite's visual mass

Part 2: Player Script

Attach Script

- Select Player node
- Click Attach Script icon (scroll icon)
- Accept default path: `res://player.gd`
- Uncheck Template, Click Create

Variable Declarations

Architecture Note: Strong Typing

GDScript supports optional static typing. This course requires type annotations for:

- All member variables
- All function parameters and return types

Benefits: Earlier error detection, better autocomplete, clearer intent.

```
1 extends Area2D
2
3 # Custom signals for decoupled communication
4 signal pickup
5 signal hurt
6
7 # Exported variables appear in Inspector
8 @export var speed: int = 350
9
10 # Internal state
11 var velocity: Vector2 = Vector2.ZERO
12 var screensize: Vector2 = Vector2(480, 720)
```

Key Concepts:

- `@export`: Makes variable Inspector-editable
- `signal`: Declares events this node can emit
- `Vector2.ZERO`: Shorthand for `Vector2(0, 0)`

Movement Implementation

```

1 func _process(delta: float) -> void:
2     # Get normalized input vector (-1 to 1 on each axis)
3     velocity = Input.get_vector("ui_left", "ui_right",
4                                 "ui_up", "ui_down")
5
6     # Update position with delta-independent movement
7     position += velocity * speed * delta
8
9     # Clamp to screen boundaries
10    position.x = clamp(position.x, 0, screensize.x)
11    position.y = clamp(position.y, 0, screensize.y)
12
13    # Animation state machine
14    if velocity.length() > 0:
15        $AnimatedSprite2D.animation = "run"
16    else:
17        $AnimatedSprite2D.animation = "idle"
18
19    # Flip sprite based on horizontal direction
20    if velocity.x != 0:
21        $AnimatedSprite2D.flip_h = velocity.x < 0

```

Design Pattern: Delta-Independent Movement**Why multiply by delta?**

- Godot targets 60 FPS (delta approximately 0.016s)
- Frame rate fluctuates due to CPU load
- speed times delta = pixels per second (frame-independent)
- Without delta: movement speed tied to frame rate

Formula: distance = velocity times time

Game State Control

```

1 func start() -> void:
2     """Reset player for new game."""
3     set_process(true)
4     position = screensize / 2
5     $AnimatedSprite2D.animation = "idle"
6
7 func die() -> void:
8     """Handle player death."""
9     $AnimatedSprite2D.animation = "hurt"
10    set_process(false)

```

Architecture note: set_process(false) disables _process() calls. Main scene controls lifecycle.

Collision Detection Setup

```
1 func _on_area_entered(area: Area2D) -> void:
2     if area.is_in_group("coins"):
3         area.pickup()
4         pickup.emit()
5     if area.is_in_group("obstacles"):
6         hurt.emit()
7         die()
```

Connecting the signal:

1. Select Player node
2. Click Node tab (next to Inspector)
3. Find area_entered signal
4. Double-click, click Connect
5. Godot creates _on_area_entered() function

Design Pattern: Groups for Type Checking

Instead of checking node types directly:

- Use is_in_group() for flexible categorization
- Allows multiple objects in same group
- Decouples player from specific node types
- Easy to add new collectible/obstacle types

Testing the Player

Run the Scene

- Press F6 or click Run Current Scene
- Use arrow keys to move
- Verify sprite flips correctly
- Check animations switch (run/idle)

Common Pitfalls

Player not moving?

- Check speed value in Inspector
- Verify Input Map has ui_left/right/up/down actions
- Confirm _process() has correct spelling

Player leaving screen?

- Verify screensize matches window settings
- Check clamp() lines for typos

Animation not playing?

- Confirm Autoplay on Load is set for idle
- Check animation names match code exactly (case-sensitive)

Complete Player Script

```
1  extends Area2D
2
3  signal pickup
4  signal hurt
5
6  @export var speed: int = 350
7
8  var velocity: Vector2 = Vector2.ZERO
9  var screensize: Vector2 = Vector2(480, 720)
10
11
12 func _process(delta: float) -> void:
13     velocity = Input.get_vector("ui_left", "ui_right",
14                                 "ui_up", "ui_down")
15     position += velocity * speed * delta
16     position.x = clamp(position.x, 0, screensize.x)
17     position.y = clamp(position.y, 0, screensize.y)
18
19     if velocity.length() > 0:
20         $AnimatedSprite2D.animation = "run"
21     else:
22         $AnimatedSprite2D.animation = "idle"
23
24     if velocity.x != 0:
25         $AnimatedSprite2D.flip_h = velocity.x < 0
26
27
28 func start() -> void:
29     set_process(true)
30     position = screensize / 2
31     $AnimatedSprite2D.animation = "idle"
32
33
34 func die() -> void:
35     $AnimatedSprite2D.animation = "hurt"
36     set_process(false)
37
38
39 func _on_area_entered(area: Area2D) -> void:
40     if area.is_in_group("coins"):
41         area.pickup()
42         pickup.emit()
43     if area.is_in_group("obstacles"):
44         hurt.emit()
45         die()
```

Architecture Review

What We Built

- Self-contained player object (scene composition)
- Signal-based communication (decoupling)
- State control via lifecycle methods (start/die)
- Group-based collision detection (flexibility)

Key Takeaways

1. **Modularity:** Player scene works independently, testable in isolation
2. **Separation of Concerns:** Movement logic separate from collision handling
3. **Extensibility:** Easy to add new animations or collision types
4. **Type Safety:** Strong typing catches errors at edit-time

Next Session Preview

- Build Coin scene with pickup behavior
- Create Main scene with spawning system
- Implement HUD with signal-driven updates
- Complete game loop architecture

This activity is to be accomplished in class.